Partie 2:

ASPECTS AVANCÉS DES CLASSES

Généricité

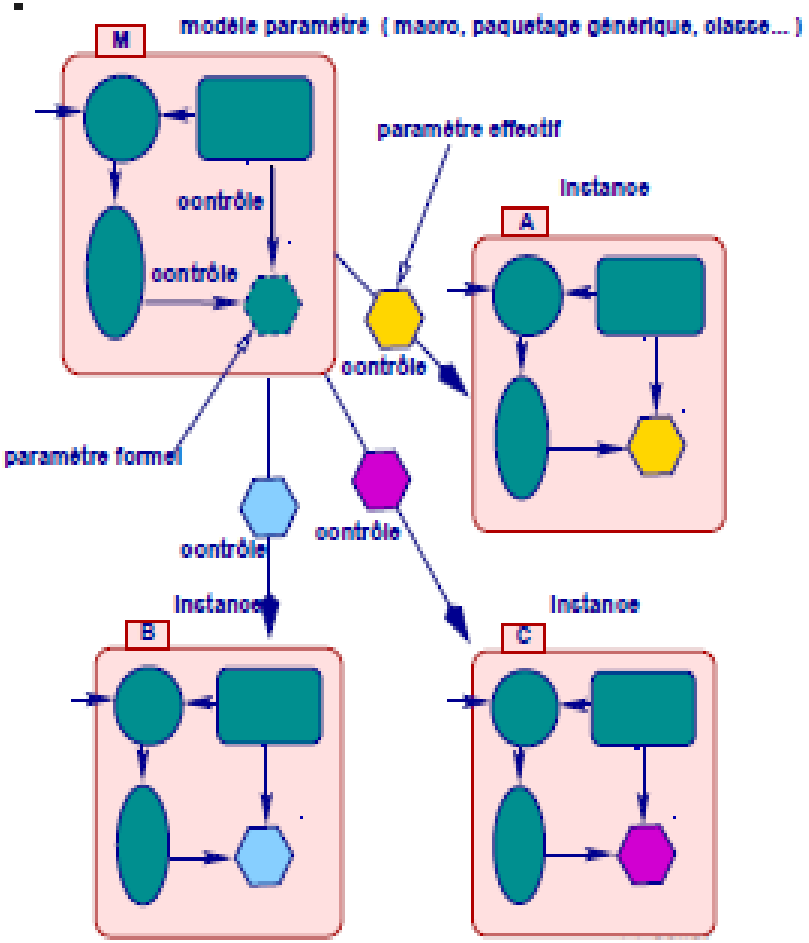
En programmation, la généricité signifie un paramétrage subtil sur des types

Quand on paramètre sur des aspects encore plus subtils (structures, méta modèles, etc…) on parle d'hypergénéricité

Objectifs : avoir des unités de code paramétrées sur des types

- Factorisation de code
- Génération automatique d'unités spécialisées
- Meilleure lisibilité
- Typage sans concession, statique ou dynamique

Schéma Général



1. ***Définir*** l'unité générique: modèle, classe , interface, méthode, paquetage…
2. **Engendrer des dérivations**: classe , interface, méthode,… effectives
3. **Utiliser les dérivations**

paramétrage générique

Déclaration d'un paramètre formel qui sera utilisé dans toute la classe.

- Utiliser la notation <T> juste après le nom de la classe
- Utiliser T dans le reste du code
- Convention : utiliser une lettre majuscule comme identifiant du paramètre
- T sera remplacé par la classe donnée en paramètre à l'instanciation

On peut alors limiter la généricité à une méthode particulière :

- Utiliser la notation <T> juste avant la signature de la méthode
- Utiliser T dans le reste du code de la méthode
- T sera inféré à partir du type des arguments de la méthode

Exemple 1

```
//une boite dans laquelle on peut mettre un objet
public class Box {
private Object object;
public void add(Object object) { this.object =
object; }
public Object get() { return object; }
public static void main(String[] args) {
Box integerBox = new Box();
integerBox.add(new Integer(10));
Integer someInteger = (Integer)integerBox.get();
System.out.println(someInteger);}
}
```

Classe sans utilisation de généricité

Exemple 2

Le cast est correct mais risque de provoquer une erreur si on ajoute autre chose que des instances de Integer

```
public class BoxDemo2 {

public static void main(String[] args) {

Box integerBox = new Box();

integerBox.add("10"); // le type est
                //maintenant String

Integer someInteger = (Integer)integerBox.get();

System.out.println(someInteger);}

}
```

Exemple 3

```
// Generic version de la classe Box .
public class Box<T> {
private T t; // T stands for "Type"
public void add(T t) { this.t = t; }
public T get() { return t; }
public static void main(String[] args) {
Box<Integer> integerBox = new Box<Integer>();
integerBox.add(new Integer(10));
Integer someInteger = integerBox.get(); // no cast!
System.out.println(someInteger);
Box<String> valS = new Box<String>("TOTOTOTO");
Box<Float> valF = new Box<Float>(12.2f);
Box<Double> valD = new Box<Double>(12.202568);
}
}
```

Généricité multiple

```
public class Paire<T, S> {
private T valeur1;
private S valeur2;
public Paire(){ this.valeur1 = null;
this.valeur2 = null; }
public Paire(T val1, S val2){ this.valeur1 = val1;
this.valeur2 = val2; }
public T getValeur1() { return valeur1; }
public void setValeur1(T valeur1) { this.valeur1 =
valeur1; }
public S getValeur2() { return valeur2; }
Public void setValeur2(S valeur2) { this.valeur2 =
valeur2; }
public String toString(){return getValeur1()+''-''+
getValeur2();
}
```

Généricité multiple

L'instanciation (ou invocation dans la terminologie Java 1.5) consiste à évaluer les paramètres formels de type (on parlera alors de paramètres réels).

Les paramètres réels ne peuvent pas être des types primitifs, mais ce défaut est compensé par l'autoboxing, qui convertit de manière transparente les valeurs des types primitifs en instances.

```
  Paire<Integer,String> p = new
Paire<Integer,String>(9,"plus grand

  chiffre");

  Integer i=p.getValeur1();

  String s=p. getValeur2();

  System.out.println(p);
```

Généricité et Héritage

Les deux mécanismes d'abstraction : généricité paramétrique et héritage peuvent se mélanger :

Sous-typage des classes pour un paramètre fixé : `PileListe<String>` est bien, un sous type de `Pile<String>`.

Combinaisons de dérivations et d'instanciations:

- Classe générique dérivée d'une classe non générique.

```
class Graphe{}
```

```
class GrapheEtiquete<TypeEtiq> extends Graphe{}
```

- Classe générique dérivée d'une classe générique.

```
class TableHash<TK,TV> extends
Dictionnaire<TK,TV>{}
```

- Classe dérivée d'une instanciation d'une classe générique.

```
class Agenda extends Dictionnaire<Date,String>{}
```

Instaciation avec joker

Elles permettent de relâcher les contraintes sur les types paramétrés pour rendre des méthodes plus réutilisables

Faire une instanciation avec joker d'un type générique c'est donner un type avec joker (« ? » est le joker) comme argument de type

`List<? extends Number>` : est une liste dont l'argument de type est un soustype de **Number**

On ne connaît pas l'argument de type exact mais on sait qu'il est un sous-type de **Number**

Les « types » avec joker

<?> désigne un type ***inconnu***

<? extends A> désigne un type ***inconnu*** qui est ***A*** ou un sous-type de **A**

<? super A> désigne un type ***inconnu*** qui est ***A*** ou un sur-type de **A**

Un seul type à la suite de **extends ou de super**

A peut être une classe, une interface, ou même un paramètre de type formel

On dit que **A contraint le joker (**impose une contrainte sur le type inconnu)

Les collections d’objet

Une collection représente un groupe d'objets par une seule unité.

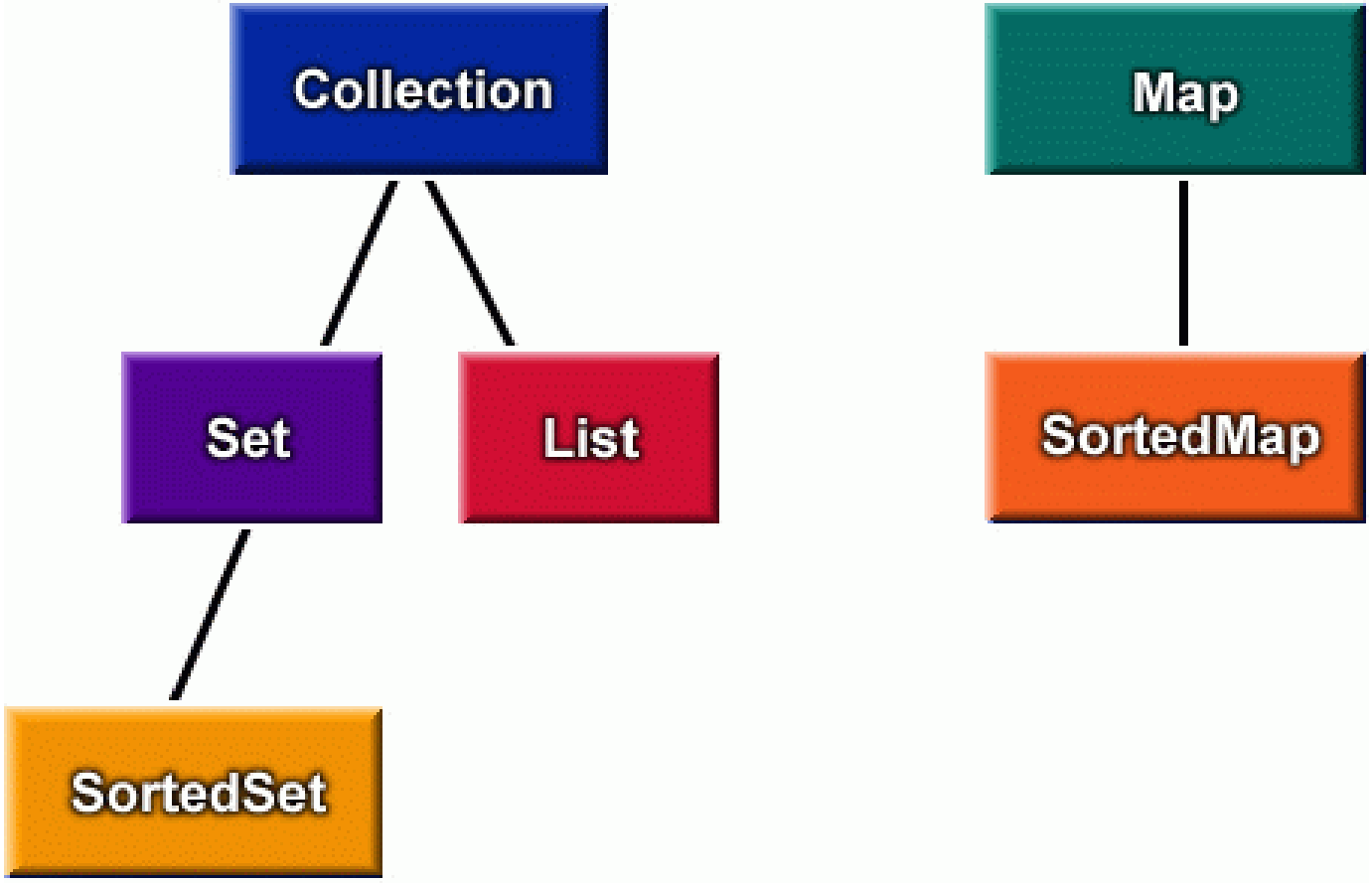
Java propose une architecture unifiée pour représenter et manipuler les collections.

Tous les Collections Framework contiennent les éléments suivants:

- Interfaces
- Implémentations
- Algorithmes

Toutes les collections en JAVA implémentent l'interface Collection par le biais de sous interfaces comme Set, Map ou List.

Hiérarchie des interfaces



Les interfaces de collections

Collection: un groupe d'objets où la duplication peut-être autorisée.

Set: est ensemble ne contenant que des valeurs et ces valeurs ne sont pas dupliquées. SortedSet est un Set trié.

List: hérite aussi de collection, mais autorise la duplication. Dans cette interface, un système d'indexation a été introduit pour permettre l'accès (rapide) aux éléments de la liste.

Map: est un groupe de paires contenant une clé et une valeur associée à cette clé. Collection traite des données simples alors que Map des données composées (clé,valeur). SortedMap est un Map trié.

L’interface Collection

Cette interface représente un minimum commun pour les objets qui gèrent des collections :

- `boolean add(E elt)`
- `void addAll(Collection<? extends E> c)`
- `void clear()`
- `boolean contains(Object obj)`
- `boolean containsAll(Collection<?> c)`
- `Iterator<E> iterator()`
- `boolean remove(Object obj)`
- `boolean removeAll(Collection<?> c)`
- `boolean retainAll(Collection<?> c)`
- `int size()`
- `Object[] toArray()`
- `<T> T[] toArray(T[] tableau)`

L’interface Set

Pour déterminer si un élément est déjà inséré dans la collection, la méthode **`equals()`** est utilisée.

Le Framework propose deux classes qui implémentent l'interface Set : **`TreeSet`** et **`HashSet`.**

Le choix entre ces deux objets est lié à la nécessité de trier les éléments :

- les éléments d'un objet **HashSet ne sont pas triés :** l'insertion d'un nouvel élément est rapide.
- les éléments d'un objet **TreeSet sont triés :** l'insertion d'un nouvel élément est plus longue.

L’interface List

Une liste est une collection ordonnée d'éléments qui autorise les doublons. La liste étant ordonnée, un élément peut être accédé à partir de son index.

L'interface List propose plusieurs méthodes pour un accès à partir d'un index aux éléments de la liste.

La gestion de cet index commence à zéro.

Pour les listes, une interface particulière est définie pour assurer le parcours dans les deux sens de la liste et assurer des mises à jour : l'interface ListIterator

Classes concrètes d'implantation

		Classes d'implantations			
		Table de hachage	Tableau de taille variable	Arbre balancé	Liste chaînée
Interfaces	Set<E>	**HashSet<E>**		TreeSet<E>	
	List<E>		ArrayList<E>		LinkedList<E>
	Map<K,V>	HashMap<K,V>		TreeMap<K,V>	

L’interface Map

Ce type de collection gère les éléments avec deux entités : une clé et une valeur associée.

La clé doit être *unique donc il ne peut y avoir de* doublons. En revanche la même valeur peut être associée à plusieurs clés différentes.

Les collections qui implémentent cette interface ne peuvent contenir des doublons.

Les collections qui implémentent cette interface utilisent une association entre une clé et une valeur.